PetStay M1

Project Overview

PetStay M1 is a Python reservation system for a single pet boarding facility. The facility has separate boarding areas and nightly capacity pools for cats and dogs.

The system supports:

1. Checking nightly availability
2. Creating reservations
3. Preventing overlapping active reservations for the same pet
4. Cancelling reservations and restoring capacity
5. Saving and loading system data through JSON

Each reservation represents one pet. A reservation stores a unique booking ID, owner ID, pet ID, pet name, pet type, check-in date, check-out date, and reservation status.

Requirements

- Python 3
- No external packages are required

How to Run

Open a terminal in the project directory and run:

```
python demo.py
```

Business Rules

1. Cats and dogs use separate boarding areas and separate capacity pools.
2. Check-in is inclusive and check-out is exclusive.
3. For example, a reservation from November 10 to November 12 occupies the nights of November 10 and November 11.
4. The check-out date must be later than the check-in date.

5. Nightly occupancy must remain between zero and the area's capacity.
6. A multi-night operation must either succeed for every night or make no changes.
7. Each booking ID must be unique.
8. The same pet cannot have overlapping active reservations.
9. Adjacent reservations are allowed when the first check-out date equals the next check-in date.
10. Different pets may reserve the same dates when sufficient capacity is available.
11. Cancelled reservations remain in the system but do not occupy capacity.
12. Each reservation may release its capacity only once.
13. Invalid operations raise specific custom exceptions and must not change the system state.

Date Convention

Dates are stored internally as `datetime.date` objects. Public service methods may accept either a `date` object or a string in `YYYY-MM-DD` format.

The stay interval follows the convention:

```
[check-in, check-out)
```

Concept Map

Line numbers refer to the integrated code.

Member 1 — Xinyi

Course concept	Code location (file:line)	Why it is used this way
Plain class, object attributes	`models.py:117–124` (`BoardingArea.__init__`)	Each area object keeps its own capacity and per-night count dict; creating the dict per instance keeps cat and dog areas independent
Encapsulation: check before modifying	`models.py:147–163` (`BoardingArea.reserve`), `models.py:165–181` (`BoardingArea.release`)	Two plain loops: the first checks every night, the second only then modifies; a failure raises before any data changes, so no partial booking survives; outside code never touches `_occupancy`
Abstract class and inheritance	`models.py:126–129` (`BoardingArea.accepts`), `models.py:234–240` (`CatArea`), `models.py:243–249` (`DogArea`)	Cat and dog areas are both boarding areas, so shared capacity/occupancy rules live in the parent; subclasses only declare `kind` and their own acceptance rule; Python refuses to instantiate the ABC (demo check "abstract class cannot be instantiated")
Polymorphism	`services.py:62–75` (`BookingService.availability`), `demo.py:38–203` (`demo_xinyi`)	Call sites use one shared interface instead of `isinstance` or subclass-name branches; the per-subclass rule `pet_type == "cat"` stays inside each subclass
Custom business exceptions	`models.py:11–12` (`BookingError`), `models.py:17–18` (`InvalidCapacityError`), `models.py:21–22` (`InvalidDateError`), `models.py:25–26` (`AreaFullError`), `models.py:29–30` (`InsufficientOccupancyError`), `models.py:33–34` (`UnknownPetTypeError`)	Named errors (invalid capacity / invalid date / area full / insufficient occupancy / unknown pet type) explain the failure reason and reject before data is modified; one shared module prevents same-name-different-class collisions between members; Yizhen (member 3) aggregates cancellation errors here
Plain data conversion, data copy	`models.py:183–192` (`BoardingArea.to_dict`)	Anita gets JSON-serializable data without access to the internal dict; mutating the returned copy cannot corrupt the area (demo check "copy isolation")
Strict date validation (project addition)	`models.py:67–70` (`_validate_date`), `services.py:24–40` (`_to_date`)	`datetime` passes `isinstance(value, date)` and its `isoformat()` carries a time, which would create a second occupancy key for the same night and allow overbooking; strict type checks route timed input to `InvalidDateError` (demo checks at `demo.py:184–201`)

Member 2 — Kyrie

Course concept	Code location (file:line)	Why it is used this way
Data class	`models.py:256–320` (`Reservation`)	`Reservation` groups a stay's booking ID, pet and owner details, dates, and status. The data class generates the initializer and lets the demo compare snapshots by value.
Composition	`services.py:52–55` (`BookingService.__init__`)	`BookingService` holds the boarding areas and reservation records, so one service coordinates the nightly occupancy and booking record.
Polymorphism	`services.py:78–124` (`BookingService.create_reservation`)	The service finds the right area through `accepts()` and calls `reserve()` through the common boarding-area interface.
Encapsulated state changes	`services.py:78–124` (`BookingService.create_reservation`), `models.py:147–163` (`BoardingArea.reserve`)	Creation checks overlapping stays before calling `reserve()`, which checks all nights before updating occupancy. The service stores the record only after `reserve()` succeeds.
Custom exceptions	`models.py:38–39` (`InvalidReservationError`), `models.py:42–43` (`DuplicateReservationError`), `services.py:78–124` (`BookingService.create_reservation`)	Invalid reservation fields and overlapping stays raise distinct `BookingError` subclasses so callers can handle each reason. Date, pet type, and capacity errors use the existing team exceptions.
Runnable demo	`demo.py:205–278` (`demo_kyrie`)	The demo covers A3, A4, A8, A11, A12, and A13 through `BookingService`. Rejected bookings compare both reservation records and occupancy against a snapshot.

Member 3 — Yizhen

Course concept	Code location (file:line)	Why it is used this way
Custom exceptions and inheritance	`models.py:48–49` (`ReservationNotFoundError`), `models.py:52–53` (`ReservationAlreadyCancelledError`)	`ReservationNotFoundError` and `ReservationAlreadyCancelledError` inherit from the shared `BookingError`. They distinguish a missing booking ID from an attempt to cancel the same reservation twice.
Composition	`services.py:126–150` (`BookingService.cancel_reservation`)	`BookingService` coordinates the stored `Reservation` object and the matching boarding area, so cancellation updates both the reservation status and nightly occupancy.
Polymorphism	`services.py:126–150` (`BookingService.cancel_reservation`)	The cancellation method finds the correct area through the shared `accepts()` interface instead of checking whether the object is specifically a `CatArea` or `DogArea`.

Course concept	Code location (file:line)	Why it is used this way
Encapsulated state change	`services.py:126–150` (`BookingService.cancel_reservation`)	Callers cancel through `BookingService.cancel_reservation()` rather than changing the reservation status directly. This keeps the status and occupancy synchronized.
Validate before modifying	`services.py:126–150` (`BookingService.cancel_reservation`), `models.py:165–181` (`BoardingArea.release`)	The method checks that the reservation exists and is still active before releasing occupancy. The area verifies all nights before modifying them, and the status changes only after release succeeds.
Runnable demo	`demo.py:280–353` (`demo_yizhen`)	The demo covers successful cancellation, repeated-cancellation rejection, missing-ID rejection, restored availability, and rebooking after cancellation through scenarios A5, A6, A7, and A14.

Member 4 — Anita

Course concept	Code location (file:line)	Why it is used this way
Custom exceptions and inheritance	`models.py:58–59` (`SaveFileError`), `models.py:62–63` (`InvalidSaveDataError`), `services.py:170–187` (`BookingService.save`), `services.py:189–201` (`BookingService.load`)	Both inherit from the shared `BookingError`. `SaveFileError` means the file itself cannot be read or written; `InvalidSaveDataError` means the content is not JSON or breaks a business rule. The demo catches each one by name.
Exception handling and chaining	`services.py:170–187` (`BookingService.save`), `services.py:189–201` (`BookingService.load`), `models.py:195–228` (`BoardingArea.from_dict`)	Low-level `OSError` and `ValueError` are caught and re-raised as team exceptions with `raise ... from error`, so callers handle one business error while the original cause stays visible. A bad capacity on load reuses Xinyi's constructor check and is turned into `InvalidSaveDataError`.
File I/O with context managers	`services.py:170–187` (`BookingService.save`), `services.py:189–201` (`BookingService.load`)	`with open(...)` closes the file even when writing or parsing fails. Saving writes to a temporary file and only then calls `os.replace`, so a failed save never leaves a half-written file where the previous save was.
Data conversion for JSON	`models.py:273–284` (`Reservation.to_dict`), `models.py:72–85` (`_parse_saved_date`), `services.py:161–168` (`BookingService.to_dict`)	JSON has no date type, so dates are written as `YYYY–MM–DD` text and parsed back into `date` objects. Only the exact `isoformat()` form is accepted, so a string such as `20261110` cannot create a second key for the same night.
Class methods as restore constructors	`models.py:195–228` (`BoardingArea.from_dict`), `models.py:286–320` (`Reservation.from_dict`), `services.py:189–201` (`BookingService.load`), `services.py:203–273` (`BookingService.from_dict`)	Each class rebuilds its own objects, so `_occupancy` and `_reservations` are only set inside their own class and outside code never edits internal collections. `load` returns a new service, so an existing service is never changed by loading.

Course concept	Code location (file:line)	Why it is used this way
Inheritance and polymorphism on restore	`models.py:195–228` (`BoardingArea.from_dict`), `services.py:203–273` (`BookingService.from_dict`)	The saved `kind` selects `CatArea` or `DogArea`. Each reservation is matched to its area through `accepts()`, the same way `create_reservation` picks an area, instead of comparing class names.
Validate before modifying (load consistency)	`services.py:203–273` (`BookingService.from_dict`)	Areas, record fields, unique booking IDs, accepted pet types, same-pet overlap and nightly occupancy are all checked on local variables first. The occupancy consistency check in `BookingService.from_dict` requires every night's saved count to equal the number of active reservations covering it (scenario A15).
Data copy	`models.py:195–228` (`BoardingArea.from_dict`), `services.py:161–168` (`BookingService.to_dict`)	The loaded occupancy is stored as a copy, so changing the input dict afterwards cannot change the restored area. Saving uses the existing `to_dict()` copies and never reads the internal dicts directly.
Runnable demo	`demo.py:365–501` (`demo_anita`)	The demo saves `petstay_data.json`, loads it, compares records, occupancy and availability, then keeps booking and cancelling on the loaded service. Thirteen tampered files plus a non-JSON file, a missing file and a failed save are each rejected with a specific exception and leave the current service unchanged.

Demo Scenarios

- **Member 1 — Xinyi:** Demonstrates cat and dog area initialization, availability queries, capacity limits, and failed operations that leave no partial changes.
- **Member 2 — Kyrie:** Demonstrates successful reservation creation, full-capacity rejection, invalid-date rejection, overlapping-reservation rejection, and valid adjacent reservations.
- **Member 3 — Yizhen:** Demonstrates successful cancellation, restored availability, repeated-cancellation rejection, unknown-booking rejection, and rebooking after cancellation.
- **Member 4 — Anita:** Demonstrates saving to `petstay_data.json` and loading it back (A9: same capacities, booking IDs, statuses and availability, with `CatArea`/`DogArea` restored), continued booking and cancelling on the loaded service (A10: new IDs stay unique, duplicate and capacity checks still work, cancelled bookings occupy nothing and cannot be cancelled again), and rejection of invalid files (A15: occupancy that does not match the active bookings, plus duplicate IDs, overlapping stays, bad dates, capacities, kinds, statuses, non-JSON and missing files), each leaving the current service unchanged.

Team Contributions

Name	Contribution
Xinyi	Implemented `BoardingArea`, `CatArea`, `DogArea`, capacity management, occupancy updates, and availability queries.
Kyrie	Implemented the `Reservation` data class, reservation creation, input validation, duplicate-reservation checks, and unique booking IDs.
Yizhen	Implemented reservation cancellation, cancellation-related exceptions, capacity restoration, cancellation demo scenarios, and README integration.
Anita	Implemented JSON saving and loading (`BookingService.save`, `load`, `to_dict`, `from_dict`), `Reservation.to_dict/from_dict`, `BoardingArea.from_dict`, the load-time consistency checks, the save/load exceptions, and the A9, A10 and A15 demo scenarios. AI assistance: Claude helped implement and verify this part.